# Efficient Thinking II — An Evaluator × Search Pattern Across Games and Reasoning

## Spending capability across domains — from a solved game to language reasoning

> *Status legend:* ***[SOLID]*** *measured & checked ·* ***[PRELIMINARY]*** *first run, under-powered.*

## Abstract

Efficient Thinking I observed, in chess, that capability decomposes as **strength = evaluator × search**: a fixed evaluator (one forward pass) sets a base level, inference-time search multiplies it, and self-learning plateaus because the *learned evaluator* is the binding constraint. This paper asks whether that pattern recurs beyond chess or is specific to it. We report recurring empirical patterns observed under deliberately modest compute — not proven laws. We test it in two directions — a *simpler* solved game (Connect-4) and a *non-game* domain (LLM mathematical reasoning) — and, to make the numbers comparable across domains, we introduce **GELO**, a calibrated cross-domain capability scale. The decomposition transfers cleanly: search is a large, portable lever that scales with inference compute and then saturates against the evaluator's ceiling. Most sharply, in reasoning a perfect verifier breaks a consensus ceiling that more search cannot (**+14.2 points**), and across four self-improvement experiments the plateau never breaks — because **you cannot improve the evaluator for free; self-play converges to its own level of play, and climbing past it requires importing information (a better oracle).** The evaluator is the bottleneck, in every domain we tested.

## 1. The measurement: GELO (Generalized Elo)

Full spec: `docs/gelo.md`. Capability is a latent ability θ on one logistic model that unifies chess Elo, Bradley–Terry, and item-response theory (Rasch): P(win/solve) = 1/(1+10^(−(θ−d)/400)). We keep chess's constants (**400 GELO = 10× better; 120 GELO = one doubling**), so a chess GELO *is* a chess Elo, and add a *calibrate-first* protocol: build a reference ladder → fit ratings from a cross-table → **gate on logistic goodness-of-fit** → pin interpretable anchors → only then rate agents. GELO is what lets a chess result and a

reasoning result be stated on the same axis, read as "N× better per game/ question." - **[SOLID]** First calibration (Connect-4): logistic goodness-of-fit **0.058** (mean |predicted – observed| pairwise win-rate) → the Elo model genuinely *fits* the game; ratings are earned, not assumed. The scale is anchored random := 0. - **[SOLID] Reasoning on the same scale — two routes, one axis.** A chess rating, a Connect-4 rating, and a reasoning ability now live on **one logistic GELO axis**, via either route in `docs/gelo.md`: - *Primary — pairwise ("beat another LLM"):* five models answer the same MATH problems and a **master judge (Kimi-K2.5)** scores every head-to-head → Bradley-Terry GELO, **anchored Kimi-K2.5 := 2800** (the "GM" of the set), the small models placed below with headroom: **Qwen3.5-4B +2743 · Qwen2.5-1.5B +2562 · Qwen2.5-3B +2509 · Qwen2.5-0.5B +2297.** The 0.5B sits ~500 GELO below the frontier; the mid-models bunch within noise (50 questions). The judge agrees with the ground-truth verifier on **72%** of decisive pairs — decent, but *itself evaluator-limited*: a referee can only rank as well as it can reason, the thesis applied to the judge (and why on *checkable* tasks the verifier still beats an LLM judge). The anchor value is free (400 GELO = 10×); we set 2800 so the numbers read like chess (elite ≈ 2800, room down toward a novice floor). - *Secondary — difficulty-anchored (IRT):* the same machinery against MATH difficulty tiers gives a monotonic ladder (L1 +1273 → L5 +1712, ~+110/level) and places a model by which tier it half-solves.

## 2. Arm A — a simpler game (Connect-4)

The *simple* end of the complexity spectrum. Connect-4 is solved, so the exact solver is a perfect oracle and a depth-limited alpha-beta gives a calibrated opponent ladder. A small conv net (policy + value) is the evaluator; PUCT MCTS is the closed loop.

- **[SOLID] Calibrated ladder**: random 0 → **depth-1 +804** → depth-2..6 +954..+1070. The **first ply of tactics is the single biggest step** (+804) — larger than all five deeper steps combined. The heuristic search ladder saturates fast: diminishing returns on search depth, quantified on a calibrated axis.
- **[SOLID] Evaluator × search decomposition** (12k oracle labels): open-loop (raw net) **+644 GELO**, closed-loop (MCTS 200) **+880 GELO → search lift ≈ +236 GELO (~4× per game)** — the same order as chess's search lift. The decomposition transfers.
- **[SOLID] The lever balance is data-dependent, not intrinsic.** With only 12k labels the raw net loses even to 1-ply search, which reads as "search-dominated." But tracing the open-loop ceiling vs data — GELO **+642 (12k) → +747 (24k) → +798 (50k)** — the raw net **reaches depth-1 parity (+804) at 50k**. So the apparent search-dominance was mostly a *starved evaluator*: given data, the evaluator catches up to low-depth search. The honest statement is that the evaluator/search balance shifts with how much you have spent on the evaluator, in either direction.

## 3. Arm B — reasoning (LLM)

The conceptual mapping (chess → language):

| chess | LLM reasoning |
|---|---|
| position | partial reasoning trace |
| policy (move priors) | next-step distribution |
| **evaluation: P(win)** | **P(this reasoning is correct)** — [0,1] |
| terminal result (win/loss) | **verifier on the final answer** (perfect in math/code) |
| MCTS search | inference-time compute over reasoning paths |
| training the net | RL / fine-tune — **RLVR = AlphaZero's loop in language** |

Two structural notes that this series should name: the evaluator's output is a **calibrated P(correct)** — binary from a verifier, scalar from a reward/PRM, vote-fraction from self-consistency — and a *well-calibrated* one is the open problem. And search has **two axes** — parallel/width (best-of-N, self-consistency) and serial/depth (long chain-of-thought); o1/R1 scaled the serial axis, and RLVR *internalizes* search into the policy rather than keeping it external as AlphaZero does.

- **[SOLID] Search scales reasoning accuracy, then saturates** (Qwen-4B, GSM8K, 120 problems, 1024-token budget): greedy pass@1 = **66.7%**; self-consistency@N = 69.2 (N=1) → 73.3 (4) → **77.5 (16)** → **77.5 (32)**. Search buys **+8–11 points** over greedy — the reasoning analog of Elo-vs-sims — then **saturates at ~77.5% by N=16**. Majority vote is a *verifier-free* evaluator, and it stops helping.
- **[SOLID] The saturation is an *evaluator* ceiling, not a policy or search ceiling.** From the same samples, self-consistency (consensus) saturates at 77.5% while **oracle-best-of-N (a perfect verifier) climbs to 91.7% and is still rising at N=32** — an **evaluator-gap of +14.2 points**. The correct answer is *in the sample set* 91.7% of the time; consensus simply can't select it. So in language, exactly as in chess, **the verifier is the binding constraint** — and with a perfect evaluator, search keeps paying off well past where consensus stalls.
- **[SOLID] Evaluator-quality *gradient*** — accuracy vs a verifier of tunable per-item accuracy q (N=16): a smooth, monotonic climb from **75.0% at q=0.5 (verifier-free consensus) → 88.3% at q=1.0 (perfect verifier)** — 77.7 / 80.6 / 83.0 / 85.9 at q=0.6/0.7/0.8/0.9. Capability scales *continuously* with evaluator quality (~+2.6 points per 0.1 of verifier accuracy); there is no threshold — **every increment of a better evaluator buys capability.** The full curve behind the two-point +14.2 ablation, and the cleanest statement of the thesis in language: to go further, improve the evaluator.
- **[SOLID] Search barely improves the *evaluator — an asymmetry with the policy. Running the master judge with self-consistency (majority of N***

> ***judgments) raised its agreement with the verifier only 72.5% → 75.0% (N:1→5) — a ~+2.5-point nudge, versus the +8–11 points self-consistency buys a policy. The reason is instructive: a policy benefits from many attempts because some land on the answer and search selects them; but if the judge can't reason a problem out, repeating the judgment reproduces the same systematic error — search only helps where the evaluator is randomly uncertain, not systematically wrong. So you can partly buy back a weak policy with compute, but not a weak evaluator*** — which is exactly why the evaluator's *quality*, not its search budget, is the binding constraint.

### The efficient-thinking frontier — size vs. search in reasoning

The core efficiency question made concrete: for a fixed compute budget, spend it on a *bigger model* or on *more search over a smaller one*? We sweep a clean size ladder (Qwen2.5 0.5B/1.5B/3B) × self-consistency N on GSM8K.

| model | params | greedy | sc@4 | sc@16 |
|---|---|---|---|---|
| Qwen2.5-0.5B | 0.5B | 30.0% | 25.0% | 41.2% |
| Qwen2.5-1.5B | 1.5B | 48.8% | 50.0% | 58.8% |
| Qwen2.5-3B | 3.0B | 67.5% | 76.2% | 83.8% |

Plotting accuracy vs. compute (≈ params × N), **every small-model-plus-search point is dominated by a bigger model with less search:** 3B greedy (67.5%, compute ≈ 3) beats 0.5B@N=16 (41.2%, ≈ 8) *and* 1.5B@N=16 (58.8%, ≈ 24). So in reasoning, **base-model size dominates search on the compute-efficiency frontier** — the *opposite* of the chess result, where search on a fixed 3.45M net was the efficient lever.

The reconciliation is the thesis itself: *search extracts what the model already contains*. The chess net was already strong on its task (~2150 open-loop), so search extracted a lot; these small LLMs are too weak on GSM8K for search to rescue — you cannot self-consistency your way up from a model that rarely finds the answer at all. **Search complements a competent base; it cannot substitute for one.** "Buy search, not size" holds only above a base-competence threshold; below it, size (base capability) dominates. (Compute ≈ params × N is a coarse proxy — self-consistency N counts full generations — but the domination is large enough to survive it.)

## 4. Self-improvement — can the flywheel raise the evaluator with no external teacher?

The self-improvement idea, stated value-first: *fix the evaluator first* — distill better-than-current value/policy targets (Monte-Carlo rollouts / MCTS-backed, anchored on real terminal

outcomes) back into the net; strength follows. We tested this across a solved game and chess, five ways. **It never broke the plateau — and *why* is the result.**

- **[SOLID] Connect-4, from scratch — the evaluator improves but strength plateaus.** Self-play expert iteration lifted the evaluator (oracle value-MAE 0.48 → 0.40 → **0.355** with more search) and strength tracked it early (GELO +119 → ~+400), confirming *fix-the-evaluator-and-strength-follows* in miniature. But strength **plateaus at ~+400 GELO** (below 1-ply search) and **4× more search per move did not raise it** — it only sharpened the evaluator's calibration. More search budget is not the missing ingredient.
- **[SOLID] The plateau is a (near) seed-independent attractor.** Warm-starting Connect-4 self-play from the *supervised* +644 net does not preserve it: the first self-play iteration erodes it to +189, then it recovers to a **~+480–500 plateau** — a little above the from-scratch +400 (the seed leaves a small lasting trace) but well below the +644 it started from and the +798 achievable with data. Self-play pulls a *better* evaluator down toward its own signal quality rather than building on it — the same erosion seen in the chess self-learned test (1214→833).
- **[SOLID, negative] Chess from scratch stalled at the random floor** — open-loop Elo bounced 254–384 with no climb over 44 iters (~1,400 self-play games). Not a method failure but a data-volume wall: bootstrapping chess from *random* needs orders of magnitude more games than two Macs generate.
- **[SOLID, negative] Chess warm-start (weak-policy seed) stalled** at ~300–326 across sim/LR settings. Root cause: a **bootstrap barrier** — MCTS quality depends on the policy prior, so from a near-random prior the search cannot play well enough to generate improving targets.
- **[SOLID, negative] The proper test — chess eval-first from a *self-learned* plateau net** ( `selfplay_warm` , open-loop 1214, off the floor, self-learned) — **did not climb; it degraded** (1214 → 833 over ~50 iters). MCTS on this net's value head doesn't play far enough above 1214 to produce improving targets, so distillation erodes rather than lifts.
- **[SOLID, positive control] An external oracle breaks the plateau.** Change *only* the value target — from the self-play game outcome to an **external oracle** (a depth-6 solver value), leaving the self-play loop otherwise identical — and Connect-4 self-training **climbs past the ~+400 plateau**: GELO +265 → +539 → **+719 by iteration 60** (oracle-value MAE 0.36 → 0.31), heading toward the oracle's own level — vs ~+400 where the self-generated-target loop stalls. This is the flip side that completes the story: the same loop that plateaus at +400 on self-generated signal climbs toward the *oracle's* level once the value target carries external information. Self-play doesn't fail because the *method* is wrong; it fails because self-generated signal has no information the evaluator lacks — supply it externally and the flywheel turns. (Seed-independent: warm-starting from the +644 supervised net with the same oracle targets similarly reaches ~+676 — it is the *target source*, not the starting point, that matters.)

**Why it can't work for free (the theory the experiments support).** A Monte-Carlo rollout *does* give the true value of a position — but the true value *under the current level of play* (the model plays both sides). Training on those labels converges the evaluator to a perfectly-calibrated evaluator *of its own level* — accurate, and **capped there**: no move better than the current level ever appears in the games, so none can be learned. The *only* way rollouts push past the current level is to play them *above* it — with search — and then the per-iteration gain equals the **search-over-policy margin** (how much MCTS beats the bare net). That margin is itself bounded by evaluator quality: a mediocre evaluator yields weak search, hence a small margin, hence no climb. This is precisely AlphaZero's engine — and precisely why it needs *deep* search × *millions* of games: a large margin sustained over many iterations. At our scale the margin was too small to cross the plateau. **Self-play cannot add information the evaluator doesn't already contain; raising the ceiling requires importing it (a better oracle).**

## 5. Synthesis — what transfers, what shifts, what binds

We report these as recurring empirical patterns — consistent across the domains and scales we tested, not claimed as laws: 1. **The decomposition transfers.** *Strength = evaluator × search* holds in a solved game and in language reasoning, on one calibrated scale (GELO). Search is a large, portable lever that scales with inference compute and then **saturates against the evaluator's ceiling** — Elo-vs-sims in chess, accuracy-vs-N in reasoning, GELO-vs-MCTS in Connect-4. 2. **The lever balance shifts with spend.** Whichever currency is *starved* dominates the returns: a thin evaluator (12k Connect-4 labels; a verifier-free consensus) makes search look all-important; feeding the evaluator (50k labels; a real verifier) rebalances it. There is no domain-intrinsic split — only how much you have spent on each side. 3. **The evaluator is consistently the binding constraint** across our experiments, shown three independent ways: reasoning consensus is broken only by a better *verifier* (+14.2); Connect-4 self-training is limited by value-head quality, not search budget; chess self-improvement stalls exactly when the evaluator is too weak for search to generate improving targets. 4. **Self-improvement has a rate, and it is the search-over-policy margin.** You cannot fix the evaluator for free: self-generated signal converges to the system's own level. Climbing requires either search that plays above the current policy (bounded by the evaluator) or an external oracle that injects new information. The positive proof is the flip side of every negative here — a perfect verifier unlocks +14.2 in reasoning, and Stockfish labels carried chess to ~2800. *Training creates information; search extracts it; neither creates what an external oracle must supply.*

## Vignette — capability-per-parameter, made vivid

Asked to play raw chess against the same Stockfish ladder used in Efficient Thinking I, a **frontier general LLM (Kimi-K2.5)** scores ~56% vs a random mover, **0% vs**

**Stockfish-1320**, and frequently cannot even emit a legal move — a performance rating of **≈341 GELO** (barely above random). The 3.45M- parameter *specialist* from Paper I plays **~2150 open-loop and ~2800 with search**. A tiny, correctly- shaped evaluator beats a giant generalist by **~1,800–2,500 GELO at the generalist's own game.** Scale is not what buys task capability; the right evaluator (and search over it) is. (Measured on llm1 via the Bedrock endpoint; small sample, and part of the gap is the LLM's difficulty producing legal moves — but the order of magnitude is unambiguous.)

## Reproducibility

Code and data generators for both arms are in the repo: `games/` (Connect-4 engine, oracle, net/MCTS/eval/calibrate/self-play) and `reasoning/` (GSM8K accuracy-vs-N sweep and the evaluator-quality ablation, via `mlx_lm`). GELO's calibrator ( `games/c4_calibrate.py` ) prints the goodness-of-fit gate. Large datasets and model weights are regenerable and not committed.